Apellidos: ..... Nombre: ..... Grup: .....

Examen final IC (*Parte 2)*

- Duración del examen: 2:30 horas.
- No podéis utilizar calculadora, móvil, apuntes, etc.
- La solución del examen se publicará en Atenea mañana por la mañana.

Nota: La indicación de la puntuación de los ejercicios es sobre 10 puntos, pero esta parte del examen final solo representa 5 puntos de la nota del examen final.

Ejercicio 1 *(2.5 puntos)*

El fragmento de programa en ensamblador SISA que se muestra a continuación se ha traducido a lenguaje máquina situando la sección de datos (.data) a partir de la dirección `0x0100` y la sección de código (.text) a partir de la dirección `0xC000`. Antes de la primera instrucción que se muestra (`MOVI R5,0`) hay 150 instrucciones que no se muestran y después de la última instrucción que se muestra (`ST 0(R2),R5`) hay 20 instrucciones que tampoco se muestran.

```
        .data
                long=6
                A: .word 1,-4, 15, 0, 12
                B: .word 13, 2, 14, -2, 9
                C: .space A
                D: .space 1
        .text
                . . .
                MOVI  R5, 0
                MOVI  R1, lo(A)
                MOVHI R1, hi(A)
                MOVI  R2, lo(C)
                MOVHI R2, hi(C)
                ADDI  R0, R1, long
        loop:   LD    R3, 0(R1)
                ADDI  R1, R1, 2
                LD    R4, long(R1)
                ADD   R7, R3, R4
                ADD   R3, R3, R7
                ST    0(R2), R3
                ADDI  R2, R2, 2
                ADD   R5, R5, R3
                CMPLEU  R7, R1, R0
                BNZ   R7, loop
        endlp:  ST    0(R2), R5
                . . .
        .end
```

Una vez ensamblado y cargado el programa en la memoria y antes de comenzar su ejecución:

a) ¿A qué direcciones de memoria corresponden las etiquetas o direcciones simbólicas siguientes? *(0.2 puntos)*

`D=0x` `loop=0x`

b) ¿Cuál es la dirección de memoria y su contenido donde han quedado almacenadas las siguientes instrucciones una vez cargado el programa? *(0.6 puntos)*

Instrucción	@memoria y contenido
`MOVHI R2, hi(C)`	Mem_w[0x]=0x
`LD R4, long(R1)`	Mem_w[0x]=0x
`BNZ R7, loop`	Mem_w[0x]=0x

c) Una vez ejecutado el programa en el SISC von Neumann ¿Cuál es la dirección de la memoria donde ha escrito la última instrucción (`ST 0(R2), R5`) y cuál es su contenido? Suponed que las instrucciones que hay antes de la primera instrucción del fragmento de programa (`MOVI R5,0`) no modifican el estado del computador excepto el PC. *(0.2 puntos)*

`Memw[0x ] = 0x`

d) Si se ejecuta el código mostrado, ¿Cuántas instrucciones se ejecutan, cuántas son lentas y cuántas son rápidas en la Harvard multiciclo y cuantos ciclos tarda desde que se ejecuta la primera instrucción del código (`MOVI R5,0`) hasta que se ejecuta la última instrucción del código (`ST 0(R2),R5`), ambas incluidas? Si se ejecuta el código en las tres versiones de los computadores SISC (Harvard uniciclo, el Harvard multiciclo y el Von Neumann) ¿Cuántos ciclos tarda en ejecutarse el código en cada una se los tres computadores? ¿Cuánto tarda en ejecutarse el código en el Harvard uniciclo, en el Harvard multiciclo y en el Von Neumann suponiendo que los tiempos de ciclo son 3.000, 2000 y 1.000 u.t. respectivamente? *(0.9 puntos)*

Nº de instruc. ejecutadas = Nº instr. lentas (H. multiciclo)= Nº instr. rápidas (H. multiciclo)=

Nº de ciclos (H. uniciclo)= Nº de ciclos (H. multiciclo)= Nº de ciclos (Von Neumann)=

Tejec(Harvard uniciclo) = Tejec(Harvard multiciclo) = Tejec(Von Neumann) =

e) Supongamos que el código ha cambiado y entre la primera instrucción del bucle y la instrucción de salto hay más de 200 instrucciones.

```
.text
       . . .
loop:  LD    R3, 0(R1)
       . . .              ; suponed que hay 200 instrucciones
       CMPLEU  R7, R1, R0
       BNZ   R7, loop
endlp: ST    0(R2), R5
       . . .
.end
```

Ahora la instrucción de salto ya no puede ser directamente un BNZ ya que esta fuera del alcance de la constante que se puede codificar en el formato de instrucción de los saltos. Suponed que el PC de la instrucción BNZ vale `0x3500`, ¿Cuál sería la dirección de la instrucción anterior y posterior más alejadas a las que podríamos acceder con la instrucción de salto? *(0.2 puntos)*

@Mem instrucción anterior = 0x

@Mem instrucción posterior = 0x

f) Una posible solución a este inconveniente es la que se presenta incompleta a continuación. Completa los huecos que faltan para que el código siga funcionando aunque haya más de 200 instrucciones entre origen y el destino del salto. *(0.4 puntos)*

Recordad que la semántica del JALR es:

Sintaxis ensamblador:	`JALR Rd, Ra`
Semántica:	`PC = PC+2; tmp=Ra&(~1); Rd=PC; PC=tmp;`
Lenguaje máquina:	`0111 aaa ddd xxxxxx`

```
loop:  LD    R3, 0(R1)
       . . .            ;suponed que hay 200 instrucciones y que no usan registros adicionales
       CMPLEU  R7, R1, R0
       BZ    R7, endlp

       _______________

       _______________

       JALR  ________
endlp: ST    0(R2), R5
```

Ejercicio 2 *(1.8 puntos)*

Cada uno de los apartados pregunta sobre un ciclo concreto de la ejecución de varias instrucciones en el SISC Von Neumann. Escribid el contenido del registro IR, el contenido de la ROM_OUT y el valor de los bits de la **palabra de control** que genera el bloque **SISC CONTROL UNIT** durante el ciclo a que hace referencia cada apartado**.** Para cada apartado/fila se indica el nodo/estado de la UC en ese ciclo y la instrucción (en ensamblador) que está almacenada en el IR en ese ciclo. Podéis ver el grafo de estados de Moore de la UC en el anexo. Suponed que el contenido de todos los registros, Rk para k=0,...7, antes de ejecutarse cada instrucción es 0.

a) Indica el contenido del registro IR. Utilizad el valor 0 para los bits que sean x (solo para este apartado). *(0.3 puntos)*

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Valor del IR** (en hexadecimal)
Addr	`LDB R1,(6)R2`	0x
Bnz	`BNZ R1, -8`	0x
Ld	`LD R2, 3(R4)`	0x

Apellidos: ..... Nombre: ..... Grup: .....

b) Indica el contenido de la ROM_OUT en hexadecimal usando las conexiones en el orden que están en el anexo. Utilizad el valor 0 para los bits que sean x (solo para este apartado). *(0.6 puntos)*

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Contenido ROM_OUT** (en hexadecimal)
Addr	`LDB R1,(6)R2`	0x
Bnz	`BNZ R1, -8`	0x
Ld	`LD R2, 3(R4)`	0x

c) Indica el valor de los bits de la **palabra de control** que genera el bloque **SISC CONTROL UNIT** durante el ciclo a que hace referencia cada apartado**. Poned x siempre que no se pueda saber el valor de un bit** (ya que no podemos suponer cómo se han implementado las x en la ROM_OUT). *(0.9 puntos)*

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Palabra de Control**																		
		@A	@B	Pc/Rx	Ry/N	OP	F	P/I/L/A	@D	WrD	Wr-Out	Rd-In	Wr-Mem	Ldlr	LdPc	Byte	Alu/R@	R@/Pc	N (hexa)	ADDR-IO (hexa)
Addr	`LDB R1,(6)R2`																			
Bnz	`BNZ R1, -8`																			
Ld	`LD R2, 3(R4)`																			

Ejercicio 3 *(1.4 puntos)*

Uno de los primeros algoritmos que se aprenden en programación son los algoritmos de ordenación de vectores de elementos. Se desea implementar un algoritmo de ordenación en lenguaje SISA para ser ejecutado que el computador **SISC Von Neumann**. Se ha escogido implementar el algoritmo de ordenación de burbuja (*Bubble Sort*) por su simplicidad. *Bubble Sort* es el algoritmo de clasificación más simple (y uno de los más ineficientes) que funciona intercambiando repetidamente los elementos adyacentes si están en el orden incorrecto. Funciona recorriendo todo el vector a ordenar, revisando cada elemento de la lista que va a ser ordenada con el siguiente, intercambiándolos de posición si están en el orden incorrecto. Al terminar de recorrer los elementos, se determina si hubo algún cambio, y de haberlo, se repite el método hasta que ya no haya cambio alguno.

Por ejemplo, suponed que el vector a ordenar es el (5 1 4 2 8), entonces la secuencia de pasos seria la siguiente:

Primer pase:
(**5 1** 4 2 8) → (**1 5** 4 2 8), Aquí, el algoritmo compara los dos primeros elementos y los intercambia puesto que 5 > 1.
(1 **5 4** 2 8) → (1 **4 5** 2 8), Intercambiar puesto que 5 > 4
(1 4 **5 2** 8) → (1 4 **2 5** 8), Intercambiar puesto que 5 > 2
(1 4 2 **5 8**) → (1 4 2 **5 8**), Ahora, dado que estos elementos ya están en orden (8 > 5), el algoritmo no los intercambia.

Segundo pase:
(**1 4** 2 5 8) → (**1 4** 2 5 8)
(1 **4 2** 5 8) → (1 **2 4** 5 8), Intercambiar puesto que 4 > 2
(1 2 **4 5** 8) → (1 2 **4 5** 8)
(1 2 4 **5 8**) → (1 2 4 **5 8**)

Ahora, la matriz ya está ordenada, pero nuestro algoritmo no sabe si está completa. El algoritmo necesita una pasada completa sin ningún intercambio para saber que está ordenado.

Tercer pase:
(**1 2** 4 5 8) → (**1 2** 4 5 8)
(1 **2 4** 5 8) → (1 **2 4** 5 8)
(1 2 **4 5** 8) → (1 2 **4 5** 8)
(1 2 4 **5 8**) → (1 2 4 **5 8**)

Se ha decidido implementar el algoritmo *Bubble Sort* usando un valor booleano que indica si ha habido una permutación o no durante el pase. En el siguiente fragmento en seudocódigo podéis ver el algoritmo. El algoritmo se podría implementar ligeramente más eficiente porque a cada pase queda un elemento menos por ordenar. Pero hemos decidido no hacerlo para simplificar el algoritmo.

```
void bubble_sort (vector[0:n-1]) {
  do {
      permutacion=FALSO
      for (i=0; i<=n-2; i++)
          if (vector[i] > vector[i+1]) {
              permutacion=VERDADERO
              /* intercanviar vector[i] y vector[i+1] */
              aux=vector[i];
              vector[i]=vector[i+1];
              vector[i+1]=aux;
          }
      }
  while {permutacion==VERDADERO}
}
```

Se desea implementar en código SISA el algoritmo de ordenación de la burbuja mostrado anteriormente para que ordene un vector alojado en la zona de memoria para datos. Se garantiza que la longitud del vector a ordenar nunca tendrá más de 30 elementos. Completa el siguiente código con las partes subrayadas que faltan.

```
        LONG=12
        .data
             vector: .word 2,6,24,12,26,92,18,74,105,36,52,64
        .text
              _____ R0, lo(vector)
              _____ R0, hi(vector)
        do:   MOVI  R1,0                ;permutación
              MOVI  R2,0                ;i
              MOVI  R3, _____
              _____ R3, R3, -2
        for:  _____ R7, R2, R3
              B____ R7, ___
        if:   A____ R4, ___, ___
              A____ R4, ___, ___
              _____ _____, _____
              _____ _____, _____
              CMPLEU R7, R5, R6
              B____ R7, _____
              MOVI  R1, _____
              _____ _____, _____
              _____ _____, _____
        fif:  ADDI  R2, R2, _____
              BNZ   R2, _____
        ffor: BNZ   R1, _____
        .end
```

Apellidos: ..... Nombre: ..... Grup: .....

Ejercicio 4 *(1.2 puntos)*

Especificad el **camino crítico** (indicando la suma ordenada de los tiempos de propagación de los bloques por los que pasa) y calculad el **tiempo de ciclo mínimo** para que el computador **SISC Von Neumann** pueda ejecutar correctamente el tipo de instrucción SISA que se indica en cada apartado (este sería el tiempo de ciclo mínimo del computador si solo ejecutara instrucciones como la indicada u otras que requieran menor tiempo). No tenéis que añadir ningún porcentaje de seguridad en el cálculo del tiempo de ciclo mínimo. Suponed que los tiempos de propagación de los bloques que forman el computador son los siguientes:

T_p(ROM_Q+) = 60 u.t.

T_p(ROM_OUT) = 80 u.t.

T_p(MUX-2-1) = 50 u.t.

T_p(MUX-4-1) = 100 u.t.

T_p(DEC) = 60 u.t. // Tiempo de propagación de un decodificador de cualquier tamaño.

T_p(REG) = 100 u.t. // Tiempo de propagación de un registro.

T_p(REGFILE) = 200 u.t. // Tiempo de **lectura** del banco de registros

T_p(ALU-slow) = 800 u.t. // Tp de la ALU para las operaciones/funciones lentas: `ADD`, `SUB`, `CMP*`.

T_p(ALU-quick) = 400 u.t. // Tp de la ALU para las operaciones/funciones rápidas: cualquier otra distinta de `ADD`, `SUB`, `CMP*`.

T_{acc}(MEMORY) = 1000 u.t. // Tiempo de acceso (para la lectura o escritura) a la memoria

T_p(AND-2) = T_p(OR-2) = 20 u.t.

T_p(NOT) = 10 u.t.

El tiempo de propagación de un bloque combinacional (T_p) y el tiempo de acceso a memoria para realizar una lectura (T_{acc}) es el tiempo desde que están estables todas las entradas necesarias hasta que se estabilizan las salidas requeridas al valor correcto para las entradas aplicadas. Desconocemos como se han implementado internamente los bloques (y podría ser de forma diferente a los vistos en clase). El tiempo de escritura en el banco de registros se ha de calcular a partir de la implementación explicada en clase. Recordad que un registro con señal de carga (Ld), REGwLd, está construido con un REG y un MUX-2-1 (no os damos el esquema interno del REGwLd, porque lo tenéis que saber).

a) T_c correspondiente al nodo de **D** (decode).

T_c=

b) T_c correspondiente al nodo de **Addi**.

T_c=

c) T_c correspondiente al nodo de **Ldb**.

T_c=

Ejercicio 5 *(0.9 puntos)*

Indicad qué cambios se producen en el estado del computador SISC Von Neumann después de ejecutar cada una de las instrucciones de la tabla suponiendo que antes de ejecutarse cada una de ellas el PC vale `0x86AC`, el contenido de todos los registros pares es `0x6789` y el de los registros impares es `0x0123`, el byte contenido en todas las direcciones pares de la memoria es `0x64` y el de todas las impares es `0x32` y el contenido de todos los puertos de entrada es `1` y el de los de salida es `2`. Utilizad la notación MEM_b[0x…]=0x... y/o MEM_w[0x…]=0x... para indicar cualquier cambio en la memoria y PORTIN[0x...]=0x... para los puertos de entrada y PORTOUT[0x...]=0x... para los de salida.

Instrucción a ejecutar	Cambios en el estado del computador
`STB -11(R2), R4`	
`JALR R2, R4`	
`LD R6, 17(R3)`	

Ejercicio 6 *(2.2 puntos)*

Debido a que es frecuente en programas tener que realizar o no una operación aritmeticológica en función de si un resultado es cierto o falso, se ha decidido crear 7 nuevas instrucciones aritmeticológicas que su ejecución es condicional.

Completad el diseño del SISC Von Neumann para que pueda ejecutar, además de las 25 instrucciones originales SISA, 7 nuevas instrucciones (**`CAND`, `COR`, `CXOR`, `CADD`, `CSUB`, `CSHA`** y **`CSHL`**) que realizan operaciones similares a las instrucciones originales (`AND`, `OR`, `XOR`, `ADD`, `SUB`, `SHA` y `SHL`) pero solo cuando se cumple una condición. Y que tienen el formato y codificación, la sintaxis ensamblador y la semántica siguientes:

Codificación: `1011 aaa bbb ddd fff`
Sintaxis: `Cxxx Rd, Ra, Rb`
Semántica: `PC=PC+2; if (Ra!=0) {Rd=Rd` op_{AL} `Rb};`

Donde `Cxxx` es el mnemotécnico concreto de cada una de las 7 operaciones. op_{AL} es la operación aritmeticológica que se realiza, que está codificada en el campo `fff` y sus valores coinciden exactamente con los de las operaciones aritmeticológicas. Notad que `Rd` es a la vez el registro destino y el primer operando fuente. Por ejemplo, si queremos hacer una resta entre el registro `R6` y `R7`, dejando el resultado en `R6` solo si el contenido de `R4` es distinto de 0 (si es cierto), la instrucción sería la siguiente: **`CSUB R6,R4,R7`** y su codificación seria 1011 100 111 110 101

Veamos, a modo de ejemplo, como sería el fragmento del algoritmo de multiplicación con terminación temprana de la práctica 5 del laboratorio que multiplica `R6` por `R7`, en su forma original y usando una de las nuevas instrucciones.

Código original

```
        MOVI R5, 0
        MOVI R1, 1
        MOVI R3, -1
for:    AND  R4, R7, R1
        BZ   R4 endif
        ADD  R5, R5, R6
endif:  SHL  R6, R6, R1
        SHL  R7, R7, R3
        BNZ  R7, for
```

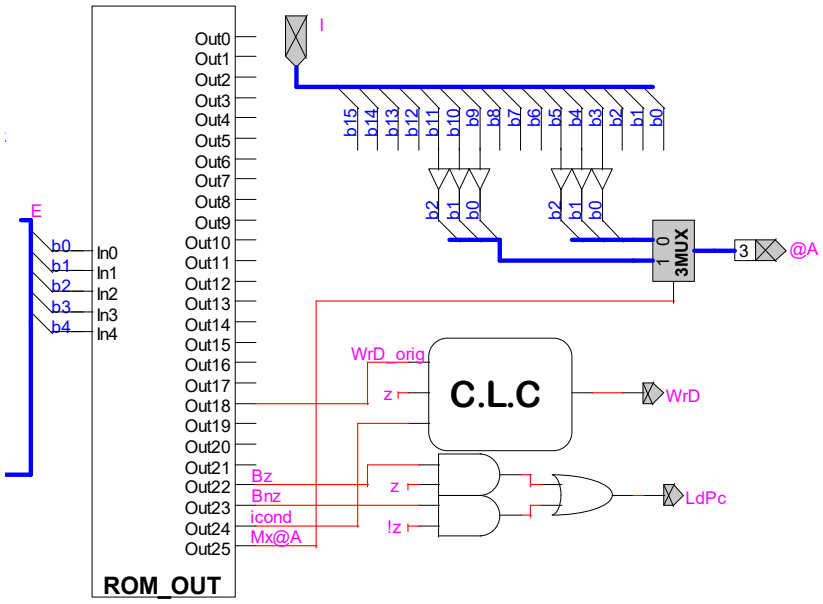
Código con la nueva instrucción

```
        MOVI R5, 0
        MOVI R1, 1
        MOVI R3, -1
for:    AND  R4, R7, R1
        CADD R5, R4, R6
        SHL  R6, R6, R1
        SHL  R7, R7, R3
        BNZ  R7, for
```

Para añadir estas nuevas instrucciones no es necesario modificar el hardware de la UPG y solo hay que realizar un par de modificaciones en el hardware de la UCG, además de modificar el contenido de las ROM's. Completad el diseño del computador para que ejecute, además de las instrucciones originales, las nuevas instrucciones `CAND`, `COR`, `CXOR`, `CADD`, `CSUB`, `CSHA` y `CSHL`.

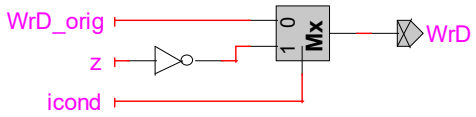
Para poder implementar las nuevas instrucciones, primero hay que modificar el hardware de la UCG ya que la señal del permiso de escritura del banco de registros (WrD) valdrá 0 o 1 durante un estado dependiendo del contenido de un registro. Para ello hemos hecho las siguientes modificaciones. Se ha añadido una señal nueva a la ROM_OUT, llamada *icond*. Esta señal nos indica que estamos en un estado de la ejecución de una instrucción condicional en el que deberíamos escribir el resultado en el banco de registros (su funcionamiento se basará en la misma idea que las señales bz y bnz). Usaremos esta señal, junto a la señal *WrD* original de la ROM_OUT y la señal z procedente de la UPG para generar la señal *WrD* para el banco de registros (el permiso de escritura).

Otra modificación que hay que hacer es permitir que la señal @A pueda escoger entre `Ra` (`IR<11..9>`) y `Rd` (`IR<5..3>`). Para ello hemos añadido un multiplexor controlado por una nueva señal *Mx@A* que también genera la ROM_OUT. En el siguiente fragmento del esquema de la UCG podemos observar las modificaciones que se han realizado (se han eliminado la mayoría de señales que no varían).



Apellidos: ..... Nombre: ..... Grup: .....

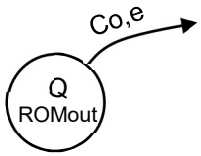
El esquema interno del circuito C.L.C. que calcula el valor de la señal *WrD* que llega al banco de registros es el siguiente.



Con las modificaciones hardware anteriores y sólo modificando el contenido de las ROM's, completad el diseño del computador para que ejecute, además de las instrucciones originales, las nuevas instrucciones.

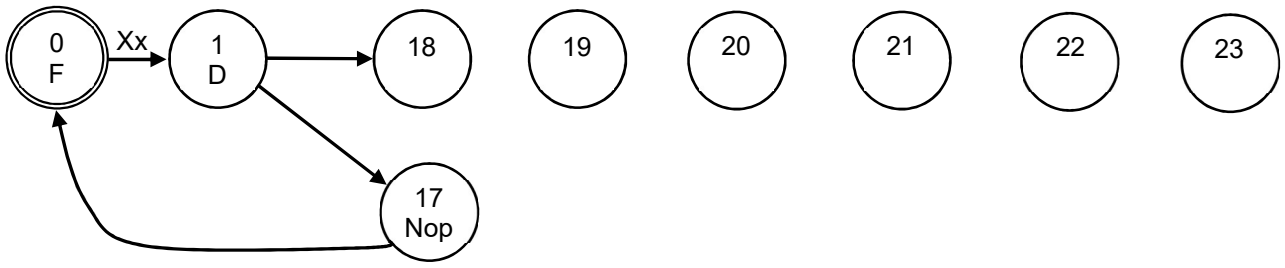
a) Completad el fragmento del grafo de estados de la figura correspondiente al circuito secuencial de la unidad de control. Se da la leyenda del grafo y los nodos necesarios para ejecutar la nueva instrucción, pero faltan arcos y etiquetas. Dibujad todos los arcos que faltan y todas las etiquetas. En número de nodos necesarios para ejecutar las nuevas instrucciones dependerá de la solución que propongáis, pero no serán más de 8 (incluyendo F y D) y deberá ser coherente con las acciones que pongáis en el apartado b). Los nodos que os sobren podéis dejarlos sin conectar. *(0.2 puntos)*

Leyenda:



CO: Código de operación de la Instrucción, I_{15} I_{14} I_{13} I_{12} (en hexadecimal)
e: Bit de extensión del código de operación (I_8)
Q: Estado (en decimal)
ROMout: Mnemotécnico de salida

Grafo:



b) Completad el contenido de la siguiente tabla que indica, mediante una fila para cada nodo del grafo de estados de la unidad de control, la acción (o acciones en paralelo) que se realiza en el computador en cada uno de los ciclos/nodos que requiere la ejecución de las nuevas instrucciones (Fetch, Decode, y los ciclos/nodos de la ejecución). Para especificar las acciones usad el mismo lenguaje de transferencia de registros que en la documentación. En número de filas/nodos de la tabla a rellenar dependerá de la solución que propongáis y deberá ser coherente con el apartado a) y c). *(0.8 puntos)*

Nodo	Mnemotécnico	Acciones
E0	F	IR ← Mem_w[PC] // PC ← PC+2
E1	D	RX ← Ra // RY ← Rb // R@ ← PC+SE(N8)*2
E18		
E19		
E20		
E21		
E22		
E23		

c) Completad (poniendo 0, 1 o x en cada bit) las filas de la tabla que especifican el contenido de la ROM_OUT para las direcciones 0 (F), 1 (D) y las de los nodos que hayáis usado para implementar la nueva instrucción (de la 18 a la 23). **Poned x siempre que el valor de un bit no importe**. *(1.0 punto)*

@ROM	**Mx@A**	**icond**	Bnz	Bz	WrMem	RdIn	WrOut	WrD	LdIr	Byte	R@/Pc	Alu/R@	Pc/Rx	Ry/N	$P/I/L/A_1$ $P/I/L/A_0$	OP_1 OP_0	MxN_1 MxN_0	MxF	F_2 F_1 F_0	$Mx@D_1$ $Mx@D_0$	***Nodo***
0																					F
1																					D
18																					
19																					
20																					
21																					
22																					
23																					

d) Completad la dirección o contenido, según corresponda, de la ROM_Q+ del SISC Von Neumann. *(0.2 puntos)*

`ROM_Q+[0x0A8]=` `0x` `ROM_Q+[` `0x` `]=0x0F`